

AMATH 482/582: HOME WORK 3

EUNICE SEUNG-HYUN HAN

Department of Applied Mathematics, University of Washington, Seattle, WA
eunich@uw.edu

ABSTRACT. This study distinguishes images of handwritten digits from the MNIST data set. Results indicate how effectively Ridge and KNN classifiers with accurate hyperparameter was able to present high performance in classifying the digits. Figures and Tables show how PCA effectively lowered the dimension which reduces computation cost but remains highly accurate reconstructed data and how the most optimal hyperparameter was chosen through cross validation for both classifiers. This study can be used to the Information Technology and the Artificial Intelligence industry as the classifier algorithm would predict the different handwriting styles that could be used language processing.

1 Introduction and Overview

Every individual has their own particular handwriting style. This report aims to distinguish images of handwritten digits from the MNIST data set by applying the Ridge and KNN classifiers. The MNIST data set consists 60,000 training images and 10,000 testing images of number digits of 0 to 9, while each image having 28×28 pixels in grayscale. Through using the PCA analysis and cross validation it allows to identify and provide the most accurate hyperparameter which is the α value and k number of neighbors of each Ridge and KNN Classifiers for high prediction of the test data. By implementing the classifiers, the report demonstrates which classifier method would show a more high accurate prediction in distinguishing images of handwritten digits.

2 Theoretical Background

The Singular Value Decomposition is a matrix factorization mathematical technique used in dimensionality reduction. For any $X \in \mathbb{R}^{n \times m}$:

$$(1) \quad X = U\Sigma V^T$$

- X is the X_{train} data that is the matrix of 1500×114
- Orthonormal matrix $U \in \mathbb{R}^{n \times n}$
- Diagonal matrix $\Sigma \in \mathbb{R}^{n \times m}$ with positive entries
- Orthonormal matrix $V \in \mathbb{R}^{m \times m}$

Using SVD, the dimensions of X can be reduced by taking the approximation of the first k columns of each U, Σ, V^T . By keeping the top k singular values and its corresponding singular vectors it gives the rank- k approximation:

$$(2) \quad X_k = U_k \Sigma_k V_k^T$$

This approximation of X , in reduced dimensionality k is projected onto the top k singular vectors. Reducing the dimensions is effective and useful because most information is concentrated in the first few singular values and it would ignore smaller singular values so it will remove the noise and focus on the most significant data. In large real-world datasets, approximating the top k singular values and their corresponding singular vectors often captures the most essential information. This approach achieves high accuracy while significantly reducing computational and storage costs.

Principal Component Analysis (PCA) [1] is a technique that leverages features from the SVD (1) to reduce the dimension of the data. It identifies the principal components that capture the most important

information and variance within the dataset, allowing us to retain a significant amount of information while using less dimensions. Below shows how PCA is implemented using scikit-learn [5] in Python:

$$(3) \quad X_c = U\Sigma V^T$$

$$(4) \quad X_{PCA} = X_c V_k$$

- X_c is the centered data of X
- Modes are the columns of V_k

In terms of using the SVD expansion in Equation 4:

$$(5) \quad X_{PCA} = (U\Sigma V^T)V_k$$

$$(6) \quad = U_k \Sigma_k$$

- Coefficients in PCA-transformed space are given by $U_k \Sigma_k$

The Inverse Transform in PCA:

$$(7) \quad X_{reconstruct} = X_{PCA} V_k^T + X_{mean}$$

Ridge classification [5] is a linear classification algorithm that makes predictions by using ridge regression and minimizing the squared error with applying regularization. The X set is $X \in \mathbb{R}^{N \times (d+1)}$ where N is the number of image samples and d is the number of pixels which is 784 (28×28). X is having $(d+1)$ the first column of X would be vector on 1's which is multiplied with the β_0 term in Equation 8. It's because the intercept of the parameter β_0 allows the model to shift the decision boundary instead of the model to go through the origin which can better fit the data. The $y \in \mathbb{R}^{N \times 1}$ would be the labels for binary classification which is $y_i \in \{-1, 1\}$. To get the prediction function, f , of the model such that $f(\mathbf{x}) \approx y$ in Ridge classification, the Maximum Likelihood Estimator (MLE) finds the parameter β that fits the data on the model to maximize the likelihood with least-squares minimization. The MLE of finding β of a linear model in linear regression:

$$(8) \quad f(\mathbf{x}) = \beta_0 + \sum_{j=1}^d \beta_j x_j$$

$$(9) \quad X = \begin{pmatrix} 1 & \mathbf{x}_0^T \\ \vdots & \vdots \\ 1 & \mathbf{x}_N^T \end{pmatrix} = \begin{bmatrix} 1 & \mathbf{x}^T \end{bmatrix} \in \mathbb{R}^{N \times (d+1)}$$

where $\mathbf{x}_N$ is the feature vector on the N th sample and β is the parameter of the model f that we want. In Equation 8 and 9, we could find $f(\mathbf{x}) = X\beta$, which tells f is a linear function with parameters β used in linear regression. β_{MLE} is the least-squares solution of the Equations 8 and 9:

$$(10) \quad \beta_{MLE} = \arg \min_{\beta} \frac{1}{2\sigma^2} \|X\beta - y\|^2$$

- $\|X\beta - y\|^2$ is the sum of the squared errors between the model's prediction $X\beta$ and the y labels. Minimizing this corresponds in maximizing the likelihood.

X is very high dimensional, which increases the probability of $X^T X$ being singular. This can be fixed with regularization which is adding, $\frac{\alpha}{2} \|\beta\|_p^p$. Adding this term ensures numerical stability and prevents overfitting.

$$(11) \quad \hat{\beta} = \arg \min_{\beta \in \mathbb{R}^J} \frac{1}{2\sigma^2} \|X\beta - y\|^2 + \frac{\alpha}{2} \|\beta\|_p^p$$

This regularization helps the numerical calculations to be stable and ensures the parameter doesn't get too big when selecting for the model. In Ridge Regression, $p = 2$ and we can solve for getting the optimal β which is the parameter of the f function.

$$(12) \quad \frac{\partial}{\partial \beta} \left((X\beta - y)^T (X\beta - y) + \frac{\alpha}{2} \beta^T \beta \right) = 0$$

$$(13) \quad X^T(X\beta - y) + \alpha\beta = 0$$

$$(14) \quad \beta = (X^T X + \alpha I)^{-1} X^T y$$

Substituting back Equation 14 to $f(\mathbf{x}) = X\beta$, makes the final prediction with determining the sign of $f(\mathbf{x})$, $\hat{y} = \text{sign}(f(\mathbf{x}))$. In multi-class prediction, the final prediction would be $\hat{y} = \arg \max(X\beta)$. If $f(\mathbf{x}) > 0$, it predicts label +1 and if $f(\mathbf{x}) < 0$, it predicts label -1.

K-Nearest Neighbors (KNN) classification [5] classifies a data point into a category using the majority of the k nearest neighbor points. The KNN classification does prediction depending on the distance between the near training samples $\mathbf{x}_{train}$ in the X_{train} data and the $\mathbf{x}_{test}$, the new data point in X_{test} , that we want to predict and classify its label. The distance computation uses Minkowski distance with $p = 2$ which is the Euclidean distance formula:

$$(15) \quad d_{Euclidean}(\mathbf{x}_{test}, \mathbf{x}) = \sqrt{\sum_{j=1}^d (\mathbf{x}_{test_j} - \mathbf{x}_j)^2}$$

After computing the distance of the $d(\mathbf{x}_{test}, \mathbf{x})$ for all training sample $\mathbf{x}$, rank the $\mathbf{x}$ train samples from the shortest distance to farthest distance from $\mathbf{x}_{test}$. Then with the chosen number of neighbors k , select the k samples that has the shortest distance from $\mathbf{x}$ to $\mathbf{x}_{test}$. The label of $\mathbf{x}_{test}$ is predicted based on the labels of these k nearest neighbors which is voting the majority labels of k neighbors by its distance to the test sample we want to predict the label of.

Cross validation [5] is a method used for selecting the value of hyperparameters which are numbers that are set before training begins and that controls the learning process. Cross validation determines the accurate hyperparameters value that shows high accuracy in prediction which would be α in Ridge Classifier and k number of neighbors in KNN Classifier. Cross validation successfully checks if the model is working well before running the new test set. The following are the steps of Cross validation [2]:

- (1) Randomly shuffle the data.
- (2) Split the data into equal sized subsets, n folds.
- (3) Train the $n - 1$ folds subsets on the model (Models of Ridge, KNN classifiers).
- (4) Using the single fold that isn't trained, validate the model by the accuracy score model fit metric.
- (5) Repeat steps 2-4 for n times with different single fold validation on each n iterations.
- (6) Get the average accuracy over folds of the n evaluations of the model fit metric.

For different hyperparameter α values in Ridge Classification [5], each α value would get a corresponding cross validation score in step (6). By comparing which α value would get the highest cross validation score, cross validation effectively demonstrates the most accurate α value. Likewise, for different hyperparameter k number of neighbors in KNN Classification [5], the k neighbors that have the highest cross validation score would be determined to be the most accurate number of k neighbors to the model.

3 Algorithm Implementation and Development

The algorithms that were used are the PCA, Ridge classifier, KNN classifier, and Cross validation. The PCA [1] class from scikit-learn [5] was applied to X_{train} , where the PCA modes represent spatial modes and the coefficients are time-dependent coefficients. The PCA [1] class from scikit-learn [5] does the centering of data automatically for obtaining the energy ratio in each k modes. Performing the PCA analysis on the first 16 PC modes of the digit images in the train set in Task 1, provides the digit images which is visualized in Figure 1. In Task 2, by computing the cumulative variance ratio of the PCA of the X_{train} data, it provides the most accurate k value for the number of modes to approximate 85% of the energy. In Task 4-6, PCA [1] allows to project the X_{train} and X_{test} onto the k PC modes with the same PCA components received in Task 2 which improves computational efficiency using fewer dimensions while obtaining a high approximation.

The Ridge classifier [5] and the KNN classifier [5] are employed in Task 4-6 to perform classification if the test image samples are able to be distinguished from the train samples of each prediction model. A function is built to get the accuracy of the prediction on each classifier for the subsets from Task 3 and for all the

digits for Task 6. The function first applies PCA to both $X_{subtrain}$ and $X_{subtest}$ which would reduce the dimensions of the data while keeping the information of the data approximately up to 85%. To obtain the most accurate Ridge classifier model, there is a for loop to determine the hyperparameter α value through cross validation [2, 5]. It iterates through many α values from $[10^{-10}, 10^{14}]$ by every multiple of 10 and each α value is implemented in the Ridge classifier algorithm from scikit-learn [5]. Then fit the PCA [1] of $X_{subtrain}$ and the labels $y_{subtrain}$ to that model and get the cross validation score using scikit-learn [5] to check if the α value in that iteration provides a high accurate prediction in classification. The cross validation algorithm [5] in this report uses 5 folds because larger numbers would present an expensive computational cost with no significant improvements in predictions and smaller numbers could show unstable prediction and lead to overly optimistic bias. $n = 5$ folds shows the balance between computational cost and reliable prediction.

After the for loop gets a list of cross validation scores for each α values, the α value that has the most highest score would be identified. This hyperparameter α would best fit into the model in Ridge classification [5]. The function returns the accuracy of the prediction of the train and test samples for using Ridge classifier using that α value that is identified. Ridge classifier implements binary classifiers one-vs-one in predicting the labels as mentioned in Section 2 for Task 4-6, but especially in Task 6 for multi-class classification, Ridge classifier in scikit-learn [5] applies the one-vs-rest strategy which fits one binary classifier per digit so for each digit there is a separate classifier that is trained to distinguish the other digits.

The process of obtaining the most accurate KNN classifier [5] model is similar with few differences. Inside the same function, there is another for loop to determine the hyperparameter k number of neighbors through cross validation. It iterates through many k values from $[1, 2, \dots, 14, 15]$ and each k value is implemented in the KNN classifier algorithm from scikit-learn [5]. Then fit the PCA [1] of $X_{subtrain}$ and the labels $y_{subtrain}$ to that model and get the cross validation score using scikit-learn [5] to check if the k value in that iteration provides a high accurate prediction in classification. After the for loop gets a list of cross validation scores for each k values, the k value that has the most highest score would be identified. This hyperparameter k would best fit into the model in KNN classification [5]. The function also returns the accuracy of the prediction of the train and test samples for using KNN classifier using the k value that is determined. The function returns the accuracy of the Ridge classifier and the KNN classifier for each subsets in Task 4 and 5, and also for the multi-class classification in Task 6.

The software packages employed are NumPy [3], Matplotlib [4], and Scikit-learn [5]. NumPy [3] facilitates numerical computations such as getting the average. Matplotlib [4] provides the digit image of the data and the approximated digit images that are reconstructed from k truncated PC modes. Scikit-learn [5] is a machine learning library that includes PCA, Ridge classifier, KNN classifier, and Cross validation.

4 Computational Results

Task 1:

Digit Images of the First 16 PC modes

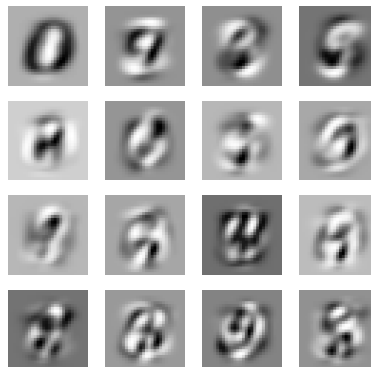


FIGURE 1. The image of the digits of the first 16 PC modes.

Figure 1 shows the first 16 PC modes as 28×28 images of the train set. As it is only taking 16 modes, it is taking a low approximation of the digit images which is the reason why the images are shown blurry.

Task 2:

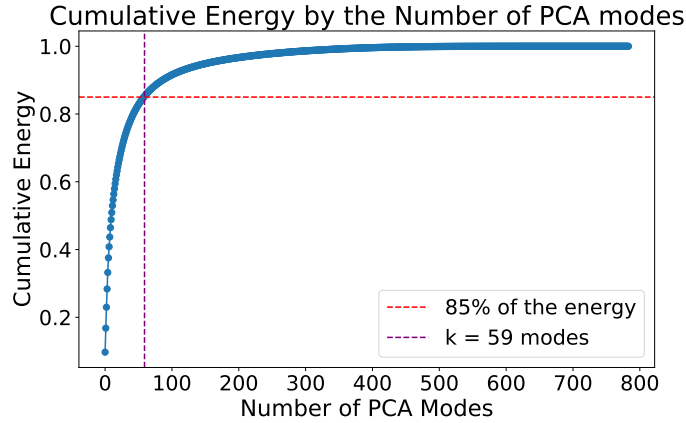
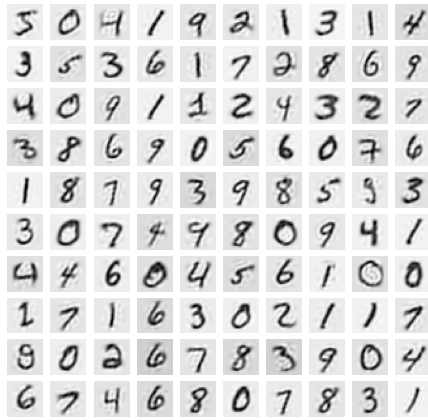


FIGURE 2. The cumulative energy by the number of PCA modes.

Approximated Digit Images Reconstructed
from 59 truncated PC modes

(A)

Original Digit Images



(B)

FIGURE 3. The original digit images and approximated digit images that is reconstructed from $k = 59$ truncated PC modes when energy exceeds to 85%.

As shown in Figure 2, the number of PC modes needed to approximate 85% of the cumulative energy is 59 modes. Figure 3a shows the approximated digit images reconstructed from 59 truncated PC modes and Figure 3b shows the original digit images. Comparing with Figure 3a with Figure 3b, it shows Figure 3a is blurry and not clear than Figure 3b but it is still showing a high quality of the digit image which explains $k = 59$ modes have successfully approximated 85% of the energy and it is clear enough to recognize the digits.

Task 4 & 5:

α	$1e-10$	$1e-5$	1	10^3	10^5	10^{10}	10^{11}	10^{12}	10^{14}
Digits									
1, 8	0.96	0.96	0.96	0.96	0.96	0.93	0.58	0.54	0.54
3, 8	0.96	0.96	0.96	0.96	0.96	0.89	0.73	0.51	0.51
2, 7	0.98	0.98	0.98	0.98	0.98	0.96	0.89	0.51	0.51

TABLE 1. The Cross validation score of each digit subset by the corresponding α value.

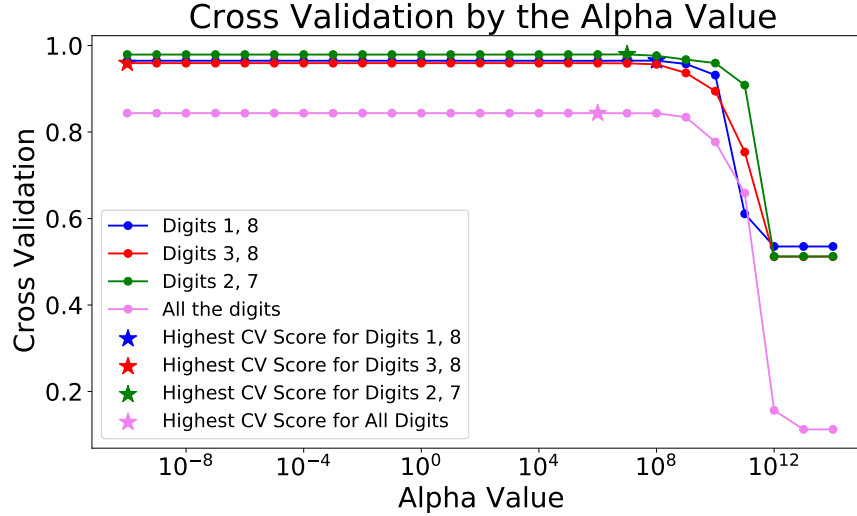


FIGURE 4. Plot of the Cross validation score by different α values in Ridge Classification.

Table 1 demonstrates the cross validation score of each digit sets, [1, 8], [3, 8], [2, 7] from [Task 4 and 5](#), by several α values that would be applied in the Ridge Classifier [5]. Figure 4 shows the α value when the cross validation score reaches the highest. For digits [1, 8]: $\alpha = 100000000$, for digits [3, 8]: $\alpha = 1e-10$, and digits [2, 7]: $\alpha = 100000$ presented the highest cross validation score, with each 0.965, 0.959, 0.98. The digit subset [2, 7] has slightly higher accuracy than [1, 8] and [3, 8] subsets and it may be because the shape of the number in [2, 7] are quite distinct to distinguish than the other subsets that has similar curves and shapes on the digit itself. These α values create the accurate model in the Ridge Classifier [5] that predicts the classification of the train and test data. As shown in Table 1 and Figure 4, when the α values gets too large it shows a rapid drop on the cross validation score as shown in Figure 4. It is because when the α values get too large, the regularization increases, which leads to high bias causing underfitting which eventually drops the cross validation score [5].

Digits	Accuracy of	
	Train	Test
1, 8	96.538%	97.914%
3, 8	95.994%	96.27%
2, 7	98.012%	97.476%

TABLE 2. The accuracy of the train and test set of using Ridge classifier on each digit set.

The accuracy of the train and test samples is obtained by the Ridge classifier model with the hyperparameter α value which got the highest cross validation score from Figure 4. Table 2 shows the accuracy of the prediction of the train and test set of each digit subsets from [Task 4 and 5](#). The results show that both the prediction of the train and test set has high accuracy regardless of the digit subsets, which demonstrates the cross validation [5] has effectively select the accurate α value and using that, the Ridge classifier [5] model is trained successfully.

Task 6:

Accuracy of	Classifier	
	Ridge	KNN
Train	84.553%	98.93%
Test	85.65%	97.59%

TABLE 3. The accuracy of the multi-class classification with Ridge and KNN classifier on each train and test samples.

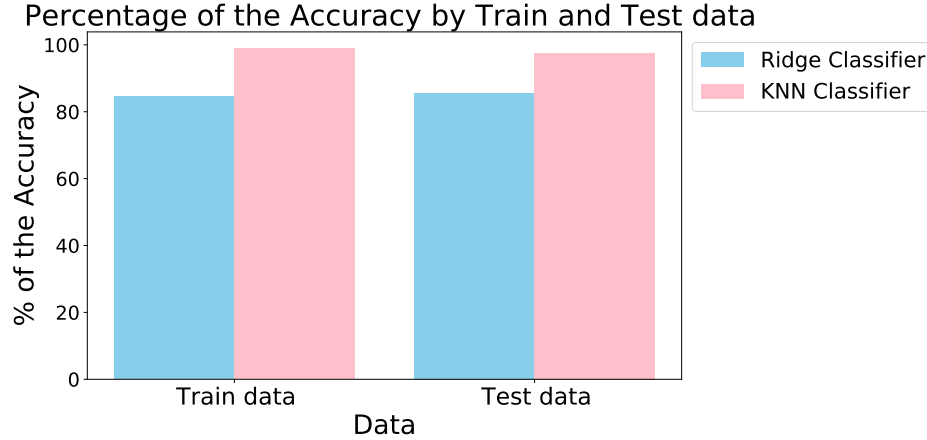


FIGURE 5. Plot of the accuracy of the train and test data for both Ridge and KNN classifiers.

The hyperparameter α that is selected by the highest cross validation score that is used in multi-class classification with Ridge classifier [5] is $\alpha = 1e-10$ with cross validation score, 0.844. And the hyperparameter k , which is the number of neighbors, that is selected by the highest cross validation score that is used in multi-class classification with KNN classifier [5] is $k = 3$ with cross validation score, 0.975. Table 3 and Figure 5 shows the accuracy of the performance of multi-class classification with the Ridge and KNN classifiers. It shows the KNN classifier [5] distinguishes images of handwritten digits from the MNIST data set much better with higher accuracy compared to the Ridge classifier [5].

The reason why the accuracy of using KNN classifier is higher than the Ridge classifier is because Ridge classifier is a linear model which cannot capture complex and nonlinear data of the digit images. Through cross validation optimal number of neighbors, $k = 3$ allows to capture local patterns of the surroundings of the data which is useful when distinguishing between digits that has a similar shape. When the k value is small it may be sensitive to noise and when it is too large it may underfit the predictions. The optimal $k = 3$ in this model would balance between focusing on local patterns with noise reduction.

5 Summary and Conclusions

The goal of this study was to train classifiers to distinguish images of handwritten digits from MNIST data set. By using Principal Component Analysis (PCA)[1], the train data achieves high approximation with lower dimensionality which reduces the computational cost while maintaining high clarification as presented in Figure 3a. By training the Ridge and KNN classifier [5] using the optimal hyperparameter obtained from the cross validation [5], the study was able to receive the best fitted model and the accuracy of each classifier that identifies which classifier was suitable in distinguishing images of handwritten digits. Figure 5 shows the accuracy of predicting the train and test sample using both the Ridge and KNN classifiers and that KNN classifier [5] presented better in this study.

Furthermore, this study can be extended to develop a system that distinguishes handwritten digits and also words, sentences, and writings by using the data of 28×28 pixel for each image sample. In the future, this study could contribute valuable and enhanced resources to the Information Technology and the Artificial Intelligence industry as the classifier algorithm would predict the handwriting of every individual that could be a power tool for language processing such as automation and assistant.

Bonus:

SVM classifier [5] is a machine learning algorithm that finds the optimal hyperplane that separates the data with maximum margin when classifying its labels. This study uses the Radial Basis Function ($\exp(-\gamma||x - x'||^2)$) kernel which is useful in classifying non linear relationship data. This study uses the regularization parameter $C = 1$ which is balancing between having a wide margin while having high accuracy. By using the kernel coefficient as $\text{scale} = \frac{1}{\text{number of samples} \times \text{X.var()}}$ this automatically adjusts the kernel coefficient based on the variance and size of the data which maintains a reasonable stable setting.

Classifier \ Accuracy of	Ridge	KNN	SVM
Train	84.553%	98.93%	99.19%
Test	85.65%	97.59%	98.41%

TABLE 4. Accuracy of the multi-class classification with Ridge, KNN, and SVM classifier.

Table 4 shows SVM classifier [5] is performing similar accuracy with KNN classifier, but with a slightly higher accuracy. The reason why SVM classifier is showing the highest accuracy in distinguishing images of handwritten digits is because SVM can leverage different function such as Gaussian Radial basis function in here, which handles high dimensional spaces well where the classes can be separated easily and effective capture complex, nonlinear data of the pixel data while Ridge regression uses a linear model 8 and the KNN classifier captures the local patterns of the data.

Acknowledgements

The author is thankful to the TA Rohin Gilman for going over the general information of the homework and explaining the mathematical concept of the Ridge Classifier and how to get the most accurate alpha value for Ridge classifier and the number of neighbors in KNeighbors classifier.

This report incorporates the theoretical background of the SVD and PCA from the author's previous homework report [Homework 2 / Page 1-2], which expanded and refined the objectives in explaining SVD and PCA.

References

- [1] PCA.
- [2] N. Frank. Amath 482 A Instructor Notes. Lecture 9-15: SVD and PCA, Regression and Cross Validation.
- [3] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, Sept. 2020.
- [4] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
- [5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al. Scikit-learn: Machine learning in python. *Journal of machine learning research*, 12(Oct):2825–2830, 2011.

[1][2][3][4][5]